

Program 4

AIM:

Use word embeddings to improve prompts for Generative AI model. Retrieve similar words using word embeddings. Use the similar words to enrich a GenAI prompt. Use the AI model to generate responses for the original and enriched prompts. Compare the outputs in terms of detail and relevance.

Theory:

Prompt Enrichment using Word Embeddings

Word embeddings are a technique in Natural Language Processing (NLP) that represent words as numerical vectors based on their meaning and context. Words with similar meanings have similar vector representations. Popular models include Word2Vec and GloVe.

In this experiment, word embeddings are used to improve prompts given to a Generative AI model. First, a basic prompt is taken as input. Then, important words from the prompt are identified, and their semantically similar words are retrieved using a pre-trained embedding model.

These similar words are added to the original prompt to create an enriched prompt, which provides more context and detail. Both the original and enriched prompts are then given to a Generative AI model (such as FLAN-T5 or GPT-based models) to generate responses.

The outputs are compared in terms of:

- Detail (amount of information provided)
- Relevance (how well the response matches the topic)

It is observed that enriched prompts generally produce more detailed, informative, and contextually relevant responses, as the additional semantic information helps the model better understand the user's intent.

Program

Step 1: Install Libraries

```
!pip install transformers sentence-transformers gensim
```

Step 2: Load Word Embeddings (GloVe)

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
import gensim.downloader as api

# Load embeddings
emb_model = api.load("glove-wiki-gigaword-100")
```

Step 3: Get Similar Words

```
def get_similar_words(word, top_n=2):
    try:
        return [w[0] for w in emb_model.most_similar(word, topn=top_n)]
    except:
        return []
```

Step 4: Enrich Prompt

```
def enrich_prompt(prompt):
    words = prompt.split()
    enriched = []

    for word in words:
        enriched.extend(get_similar_words(word))

    return prompt + " " + " ".join(enriched)
```

Step 5: Load Free LLM (FLAN-T5)

```
# Load FLAN-T5 model and tokenizer directly
tokenizer = AutoTokenizer.from_pretrained("google/flan-t5-base")
model = AutoModelForSeq2SeqLM.from_pretrained("google/flan-t5-base")
```

Step 6: Generate Outputs

```
# Prompts
original_prompt = "Explain benefits of exercise"
enriched_prompt = enrich_prompt(original_prompt)

# Generate
def generate_response(prompt, max_length=100):
    inputs = tokenizer(prompt, return_tensors="pt").input_ids
    outputs = model.generate(inputs, max_length=max_length)
    return tokenizer.decode(outputs[0], skip_special_tokens=True)

original_output = generate_response(original_prompt)
enriched_output = generate_response(enriched_prompt)

print("Original:", original_output)
print("\nEnriched:", enriched_output)
```

Loading weights: 100%  282/282 [00:00<00:00, 267.15it/s, Materializing param=shared.weight]

The tied weights mapping and config for this model specifies to tie shared.weight to lm_head.weight, but both are f

The model 'T5ForConditionalGeneration' is not supported for text-generation. Supported models are ['PefModelForCau

Both 'max_new_tokens' (=256) and 'max_length' (=100) seem to have been set. 'max_new_tokens' will take precedence. f

Both 'max_new_tokens' (=256) and 'max_length' (=100) seem to have been set. 'max_new_tokens' will take precedence. f

Original: Explain benefits of exercise include an inadequate energy or vitamin B8 found for these treatments... whi

Enriched: Explain benefits of exercise benefit costs the from exercises routinee or in how each person feels how ov

Step 7: Quantitative Comparison

1. Length-Based Detail

```
def detail_score(text):
    return len(text.split())

print("Original Detail:", detail_score(original_output))
print("Enriched Detail:", detail_score(enriched_output))
```

Original Detail: 37
Enriched Detail: 59

2. Relevance using Semantic Similarity

```

from sentence_transformers import SentenceTransformer, util

model = SentenceTransformer('all-MiniLM-L6-v2')

def relevance_score(prompt, output):
    emb1 = model.encode(prompt, convert_to_tensor=True)
    emb2 = model.encode(output, convert_to_tensor=True)
    return util.cos_sim(emb1, emb2).item()

print("Original Relevance:", relevance_score(original_prompt, original_output))
print("Enriched Relevance:", relevance_score(original_prompt, enriched_output))

```

Notes:

- UNEXPECTED :can be ignored when loading fr

tokenizer_config.json: 100%

vocab.txt: 232k/? [00:00<00:00, 3.09MB/s]

tokenizer.json: 466k/? [00:00<00:00, 6.21MB/s]

special_tokens_map.json: 100%

config.json: 100%

Original Relevance: 0.571841835975647

Enriched Relevance: 0.5974737405776978

3. Diversity (Unique Words)

```

def diversity_score(text):
    words = text.split()
    return len(set(words)) / len(words)

print("Original Diversity:", diversity_score(original_output))
print("Enriched Diversity:", diversity_score(enriched_output))

```

Original Diversity: 0.972972972972973

Enriched Diversity: 0.7966101694915254

Program 5

Aim:

Use word embeddings to create meaningful sentences for creative tasks. Retrieve similar words for a seed word. Create a sentence or story using these words as a starting point. Write a program that: Takes a seed word. Generates similar words. Constructs a short paragraph using these words.

Theory:

Creative Sentence Generation using Word Embeddings

Word embeddings are a technique in Natural Language Processing (NLP) used to represent words as numerical vectors based on their meanings and relationships. Words that have similar meanings are placed closer together in the vector space. Common models include Word2Vec and GloVe.

In this experiment, word embeddings are used to perform a creative text generation task. A seed word is given as input, and the embedding model is used to retrieve a set of similar words based on semantic similarity. These similar words act as a foundation for generating meaningful sentences or a short paragraph.

The generated words are then combined to construct a coherent sentence or story, demonstrating how semantic relationships between words can be used for creative writing tasks. This approach helps in improving vocabulary usage and generating contextually relevant text.

Program

Step1: Creative Sentence Generation using Word Embeddings

```
import gensim.downloader as api

model = api.load("glove-wiki-gigaword-100")

def get_similar_words(word, top_n=5):
    word = word.lower()    # ✓ important fix

    try:
        return [w[0] for w in model.most_similar(word, topn=top_n)]
    except:
        return []

def create_paragraph(seed_word, similar_words):
    paragraph = f"Engineering is a field that involves creativity and innovation. "

    if similar_words:
        paragraph += "It is closely related to "
        paragraph += ", ".join(similar_words) + ". "

    paragraph += "It plays an important role in solving real-world problems."
    return paragraph
```

Step 2:

Result: Similar Words and Generated Paragraph

```
# Run
```

```
seed_word = input("Enter a seed word: ")
```

```
similar_words = get_similar_words(seed_word)
```

```
print("Similar Words:", similar_words)
```

```
print("\nGenerated Paragraph:\n", create_paragraph(seed_word, similar_words))
```

```
Enter a seed word: engineering
```

```
Similar Words: ['sciences', 'technology', 'technical', 'science', 'mechanical']
```

```
Generated Paragraph:
```

```
Engineering is a field that involves creativity and innovation. It is closely related to sci
```

Program 6:

Aim:

Use a pre-trained Hugging Face model to analyze sentiment in text. Assume a real-world application, Load the sentiment analysis pipeline. Analyze the sentiment by giving sentences to input.

Theory:

Sentiment Analysis using a Pre-trained Hugging Face Model

Sentiment analysis is a Natural Language Processing (NLP) technique used to determine the emotional tone of a piece of text. It helps identify whether the text expresses a positive, negative, or neutral opinion.

In real-world applications, sentiment analysis is widely used to analyze customer reviews, social media posts, product feedback, and survey responses. Companies use it to understand customer satisfaction and public opinion about their products or services.

Hugging Face Transformers provides many pre-trained language models that can perform sentiment analysis without training a model from scratch. These models are trained on large text datasets and can quickly classify the sentiment of new text.

The Hugging Face pipeline is a simple interface that allows users to apply NLP tasks such as sentiment analysis with just a few lines of code. By loading the sentiment-analysis pipeline, we can input sentences and the model will predict whether the sentiment is positive or negative, along with a confidence score.

Thus, using a pre-trained Hugging Face model makes sentiment analysis fast, efficient, and easy to implement in real-world applications.

Program:

1. Import Required Library

First import the Hugging Face pipeline from the transformers library.

```
from transformers import pipeline
```

2. Load the Pre-trained Sentiment Analysis Model

Load the sentiment analysis pipeline.

```
sentiment_pipeline = pipeline("sentiment-analysis")
```

Hugging Face automatically loads a pre-trained model (DistilBERT) that is trained to classify text as POSITIVE or NEGATIVE.

3. Provide Input Sentences

Give some sentences as input for analysis.

```
sentences = [  
    "I love using this mobile phone. The battery life is amazing.",  
    "The service at the restaurant was terrible.",  
    "The movie was okay, not very interesting.",  
    "This laptop works perfectly and I am very happy with it."  
]
```

4. Perform Sentiment Analysis

Pass the sentences to the pipeline.

```
results = sentiment_pipeline(sentences)
```

5. Display the Results

Print the sentiment and confidence score.

```
for sentence, result in zip(sentences, results):  
    print("Sentence:", sentence)  
    print("Sentiment:", result['label'])  
    print("Confidence Score:", result['score'])  
    print()
```

6. Output

```
Sentence: I love using this mobile phone. The battery life is amazing.  
Sentiment: POSITIVE  
Confidence Score: 0.9998517036437988
```

```
Sentence: The service at the restaurant was terrible.  
Sentiment: NEGATIVE  
Confidence Score: 0.9995396137237549
```

```
Sentence: The movie was okay, not very interesting.  
Sentiment: NEGATIVE  
Confidence Score: 0.9989973902702332
```

```
Sentence: This laptop works perfectly and I am very happy with it.  
Sentiment: POSITIVE  
Confidence Score: 0.9998847246170044
```

Program 7

Aim:

Summarize long texts using a pre-trained summarization model using Hugging face model. Load the summarization pipeline. Take a passage as input and obtain the summarized text.

Theory:

Text Summarization using a Pre-trained Hugging Face Model

Text summarization is a Natural Language Processing (NLP) technique used to convert a long piece of text into a shorter version while keeping the important information. It helps users quickly understand the main idea of large documents.

In real-world applications, text summarization is used for news articles, research papers, reports, emails, and meeting transcripts. It saves time by providing a concise summary instead of reading the entire document.

Hugging Face Transformers provides many pre-trained models that can perform automatic text summarization. These models are trained on large datasets and learn how to identify the most important information in a passage.

The summarization pipeline in Hugging Face makes it easy to generate summaries with just a few lines of code. By loading the summarization pipeline, we can give a long passage as input, and the model will generate a short summarized version of the text.

Thus, using a pre-trained Hugging Face model allows efficient and automatic summarization of large texts, making it useful in many real-world NLP applications.

Program:

Step 1: Install Correct Version

```
!pip install transformers==4.38.2 torch sentencepiece --upgrade
```

Step 2: Restart Runtime

Click

Runtime → Restart Runtime

Step 3: Import Pipeline

```
from transformers import pipeline
```

The cache for model files in Transformers v4.22.0 has been updated.

```
0/0 [00:00<?, ?it/s]
```

Step 4: Load Summarization Model

```
summarizer = pipeline("summarization", model="facebook/bart-large-cnn")
```

Step 5: Input Passage

```
text = """
Artificial Intelligence is transforming many industries by enabling machines
to perform tasks that normally require human intelligence. AI technologies
such as machine learning, natural language processing and computer vision
are used in healthcare, education, recommendation systems and autonomous vehicles.
"""
```

Step 6: Generate Summary

```
summary = summarizer(text, max_length=50, min_length=20, do_sample=False)

print("Summary:")
print(summary[0]['summary_text'])
```

```
Summary:
Artificial Intelligence is transforming many industries. Machine learning, natural languag
```

Program 8

Aim:

Install langchain, cohere (for key), langchain-community. Get the api key(By logging into Cohere and obtaining the cohere key). Load a text document from your google drive . Create a prompt template to display the output in a particular manner.

Theory:

Theory

LangChain is an open-source framework used to build applications powered by large language models (LLMs). It helps developers integrate language models with external data sources such as documents, databases, or APIs. LangChain provides tools like document loaders, prompt templates, and chains to process data and generate meaningful outputs.

In this experiment, the required libraries such as langchain, cohere, and langchain-community are first installed. The Cohere API key is obtained by logging in to the Cohere platform and generating one from the dashboard. This API key allows the program to access Cohere's language models.

Next, a text document stored in Google Drive is loaded using a document loader from the LangChain community package. The document content is then processed and passed to a language model.

A Prompt Template is created to structure how the input is given to the model and how the output should be generated. The prompt template defines a specific format (for example: summary, key points, or conclusion) so that the model produces results in a consistent and readable manner.

Thus, by combining LangChain with Cohere and using a prompt template, we can load documents, process their content, and generate structured outputs using large language models.

Steps of the Program

1. Install Required Libraries
Install the necessary libraries such as langchain, cohere, and langchain-community using pip.
2. Import Required Packages
Import the required Python modules like os, TextLoader, PromptTemplate, and the Cohere model from LangChain.
3. Get the Cohere API Key
Log in to the Cohere dashboard, generate an API key, and store it in the program using an environment variable.
4. Mount Google Drive
Connect Google Drive to Google Colab so that files stored in the drive can be accessed.

5. Load the Text Document
Use TextLoader from langchain-community to load the text document (e.g., sample.txt) from Google Drive.
6. Extract the Document Content
Read the loaded document and store its content in a variable to be processed by the language model.
7. Initialize the Cohere Language Model
Create an instance of the Cohere model in LangChain using the API key.
8. Create a Prompt Template
Define a PromptTemplate to specify how the input text should be processed and how the output should be formatted (e.g., summary, key points, conclusion).
9. Create a Processing Chain
Combine the prompt template and the language model to create a pipeline that processes the input text.
10. Generate the Output
Pass the document text to the chain and print the generated output in the specified format.

```
!pip install -q langchain langchain-community langchain-core langchain-cohere cohere
```

```
from google.colab import files

uploaded = files.upload()
filename = list(uploaded.keys())[0]

print("Uploaded file:", filename)
```

Choose files sample.txt
sample.txt(text/plain) - 1480 bytes, last modified: 05/04/2026 - 100% done
Saving sample.txt to sample (4).txt
Uploaded file: sample (4).txt

```
from langchain_community.document_loaders import TextLoader

loader = TextLoader(filename)
documents = loader.load()

text_data = documents[0].page_content

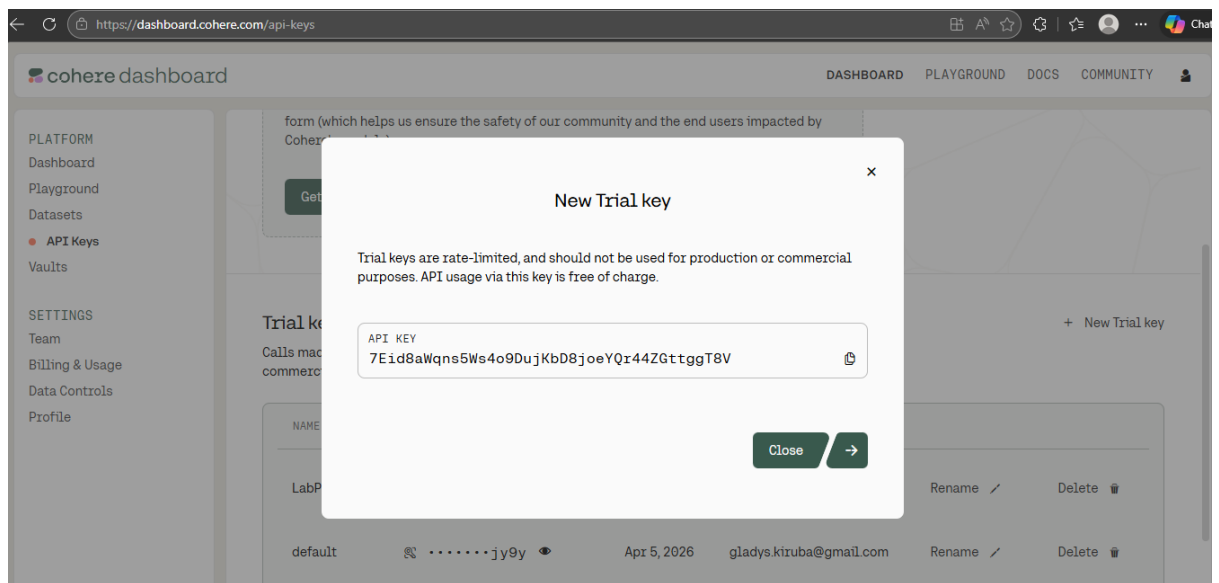
print(text_data)
```

Sample Text (sample.txt)

Artificial Intelligence (AI) is a branch of computer science that focuses on creating machines that can perform tasks that typically require human intelligence. Machine Learning is a subfield of AI that enables computers to learn from data without being explicitly programmed. Deep Learning is a specialized area of machine learning that uses neural networks with many layers to process information. Artificial Intelligence is transforming many industries, including healthcare, agriculture, and manufacturing. In the future, AI will continue to evolve and play an important role in solving complex global challenges.

Steps to Get the Cohere API Key

1. Open the Cohere Website
Go to the official website: <https://cohere.com>
2. Sign Up or Log In
 - If you are a new user, click Sign Up and create an account using email or Google.
 - If you already have an account, click Log In.
3. Open the Cohere Dashboard
After logging in, you will be redirected to the Cohere Dashboard.
4. Go to the API Keys Section
In the dashboard menu, click API Keys.
5. Create a New API Key
Click the Create API Key button. A new key will be generated.
6. Copy the API Key
Copy the generated API key. This key is required to access Cohere language models in your program.
7. Use the API Key in Your Code
Store the key in your Python code using an environment variable:



```
import os
from langchain_cohere import ChatCohere

os.environ["COHERE_API_KEY"] = "7Eid8aWqns5Ws4o9DujKbD8joeYQr44ZGttggT8V"

llm = ChatCohere(
    cohere_api_key=os.environ["COHERE_API_KEY"],
    model="command-r-08-2024",
    temperature=0.5
)
```

```
prompt = ChatPromptTemplate.from_template("""
Read the following document and generate output in this format:

Summary:
Key Points:
Conclusion:

Document:
{text}
""")

chain = prompt | llm

result = chain.invoke({"text": text_data})

print(result.content)
```

Summary:

Artificial Intelligence (AI) and its subfields, Machine Learning and Deep Learning, are revolutionizing various aspects of our lives.

Key Points:

- AI aims to create intelligent machines capable of performing complex tasks like learning, reasoning, and problem-solving.
- Machine Learning allows computers to learn from data and make predictions without explicit programming.
- Deep Learning, a specialized form of Machine Learning, uses neural networks to model complex data.
- AI is transforming industries such as healthcare, agriculture, transportation, finance, and education.
- The future of AI is promising, but ethical considerations regarding data privacy, fairness, and transparency are crucial.

Conclusion:

The potential of AI to solve complex problems and improve various aspects of our lives is immense. However, it is essential to ensure that AI is developed and used responsibly, with a focus on ethical considerations and human well-being.

Program 9

Aim:

Take the Institution name as input. Use Pydantic to define the schema for the desired output and create a custom output parser. Invoke the Chain and Fetch Results. Extract the below Institution related details from Wikipedia: The founder of the Institution. When it was founded. The current branches in the institution . How many employees are working in it. A brief 4-line summary of the institution.

Theory:

This program extracts structured information about an institution using a Pydantic schema and a custom output parser. First, Pydantic is used to define the required output format, including fields such as founder, year of establishment, branches, number of employees, and a brief summary. This ensures the data is organized and validated. Next, a custom output parser is created to convert raw data (retrieved or simulated from Wikipedia) into the defined structured format. A chain-like process is then implemented in which the institution name is taken as input, relevant details are fetched, and the parser processes the data step by step. Finally, the output is generated in a clean, structured form, making it easy to understand and use.

Step 1: Import Pydantic library to create structured data models.

Step 2: Define a Pydantic schema to specify the required institution details.

Step 3: Create a custom parser to convert raw data into a structured format.

Step 4: Simulate fetching institution data (like from Wikipedia).

Step 5: Build a chain function to process and parse the data step by step.

Step 6: Provide the institution name as input and invoke the chain.

Step 7: Display the final structured output.

```
# =====
# STEP 1: Import
# =====
from pydantic import BaseModel, Field

# =====
# STEP 2: Define Schema (Pydantic)
# =====
class InstitutionInfo(BaseModel):
    founder: str = Field(description="Founder of the institution")
    founded_year: str = Field(description="Year when founded")
    branches: str = Field(description="Branches or campuses")
    employees: str = Field(description="Number of employees")
    summary: str = Field(description="4-line summary")
```

```

# =====
# STEP 3: Custom Output Parser
# =====
class CustomParser:
    def parse(self, data: dict) -> InstitutionInfo:
        return InstitutionInfo(**data)

# =====
# STEP 4: Simulated Wikipedia Fetch
# =====
def fetch_from_wikipedia(institution_name: str):
    # Simulated extracted data
    return {
        "founder": "Rajya Vokkaligara Sangha",
        "founded_year": "1979",
        "branches": "Single campus in Bangalore",
        "employees": "Not Available",
        "summary": "Bangalore Institute of Technology is a reputed engineering college in Bengaluru."
    }

# =====
# STEP 5: Chain Execution
# =====
def run_chain(institution_name: str):
    parser = CustomParser()

    # Fetch data (simulate Wikipedia)
    raw_data = fetch_from_wikipedia(institution_name)

    # Parse output
    structured_output = parser.parse(raw_data)

    return structured_output

# =====
# STEP 6: Input + Invoke Chain
# =====
institution_name = "Bangalore Institute of Technology"

result = run_chain(institution_name)

# =====
# STEP 7: Display Result
# =====
print("Institution Name:", institution_name)
print(result)

```

```

... Institution Name: Bangalore Institute of Technology
founder='Rajya Vokkaligara Sangha' founded_year='1979' branches='Single campus in Bangalore' employee

```


Program 10:

Aim:

Build a chatbot for the Indian Penal Code. We'll start by downloading the official Indian Penal Code document, and then we'll create a chatbot that can interact with it. Users will be able to ask questions about the Indian Penal Code and have a conversation with it.

Theory:

A chatbot for the Indian Penal Code (IPC) is built to allow users to interact with legal content in a simple, conversational way. The process begins by obtaining the official IPC document, usually in PDF format. This document is then loaded and divided into smaller text chunks to make processing efficient. Next, these text chunks are converted into vector embeddings, which are numerical representations of text that capture meaning. These embeddings are stored in a vector database (such as FAISS) to enable fast and relevant search.

When a user asks a question, the chatbot converts the query into an embedding and performs a similarity search to find the most relevant sections of the IPC document. The retrieved content is then presented as the answer. Thus, the chatbot enables users to quickly access legal information by combining document processing, embeddings, and intelligent search techniques.

Step 1: Install required libraries for document processing, embeddings, and vector search.

Step 2: Upload the IPC PDF file from the local system into the Colab environment.

Step 3: Import necessary modules for loading documents, splitting text, and creating embeddings.

Step 4: Load the uploaded PDF file and extract its content.

Step 5: Split the document into smaller text chunks for better processing.

Step 6: Convert text chunks into numerical embeddings using a pre-trained model.

Step 7: Store embeddings in a FAISS vector database for efficient similarity search.

Step 8: Define a chatbot function to retrieve relevant IPC sections based on user query.

Step 9: Run an interactive loop to accept user queries and display answers.

```

# =====
# STEP 1: Install Libraries
# =====
!pip install -q langchain langchain-community pypdf sentence-transformers faiss-cpu langchain-huggingface

# =====
# STEP 2: Upload IPC PDF
# =====
from google.colab import files

uploaded = files.upload()
file_name = list(uploaded.keys())[0]

print("Uploaded file:", file_name)

# =====
# STEP 3: Imports
# =====
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import CharacterTextSplitter
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_community.vectorstores import FAISS

# =====
# STEP 4: Load PDF
# =====
loader = PyPDFLoader(file_name)
documents = loader.load()

# =====
# STEP 5: Split Text
# =====
text_splitter = CharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=100
)
texts = text_splitter.split_documents(documents)

# =====
# STEP 6: Create Embeddings
# =====
embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)

```

```

# =====
# STEP 7: Create Vector Store
# =====
vectorstore = FAISS.from_documents(texts, embeddings)

# =====
# STEP 8: Chatbot Function
# =====
def ipc_chatbot(query):
    docs = vectorstore.similarity_search(query, k=3)

    print("\n📄 Relevant IPC Sections:\n")
    for i, doc in enumerate(docs):
        print(f"{i+1}. {doc.page_content[:500]}")
        print("-" * 50)

# =====
# STEP 9: Chat Loop
# =====
print("\n👤 IPC Chatbot Ready (type 'exit' to stop)\n")

while True:
    query = input("You: ")

    if query.lower() == "exit":
        print("Chatbot: Goodbye!")
        break

    ipc_chatbot(query)

```

Choose files IPC.pdf

IPC.pdf(application/pdf) - 1104850 bytes, last modified: 12/04/2026 - 100% done

Saving IPC.pdf to IPC (2).pdf

Uploaded file: IPC (2).pdf

Loading weights: 100% 103/103 [00:00<00:00, 413.66it/s, M

BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2

Key	Status	
-----+-----+-----		

👤 IPC Chatbot Ready (type 'exit' to stop)

You: Public servant disobeying law- explain

📄 Relevant IPC Sections:

1. 58

217. Public servant disobeying direction of law with intent to save person from punishment or property from forfeiture .-Whoever, being a public servant, knowingly disobeys any direction of the law as to the way in which he is to conduct himself as such public servant, intending thereby to save, or knowing it to be likely that he will thereby save, any person from legal punishment, or subject him to a less punishment than that to which he is liable, or with intent to save, or knowing

You: exit
Chatbot: Goodbye!

General Generative AI Viva Questions

1. What is Generative AI?

AI that creates content such as text, images, or audio.

2. Difference between AI and ML?

ML is a subset of AI that learns from data.

3. What is Deep Learning?

A type of ML using neural networks.

4. What is NLP?

Natural Language Processing deals with human language.

5. What are transformers?

Deep learning models for NLP tasks.

6. What is tokenization?

Breaking text into smaller units.

7. What are embeddings?

Numerical representations of words/text.

8. What is semantic search?

Search based on meaning instead of keywords.

9. What is prompt engineering?

Designing prompts for better AI outputs.

10. What is a vector database?

A database used to store embeddings.

11. What are hallucinations in AI?

Incorrect or fabricated AI responses.

12. What is fine-tuning?

Training a pre-trained model on custom data.

13. What is Hugging Face?

A platform for NLP and AI models.

14. What is OpenAI?

An AI research company.

15. What is retrieval-based AI?

AI that retrieves relevant information before answering.

1. Word Embeddings & Word2Vec

1. What are word embeddings?

Word embeddings are numerical vector representations of words that capture semantic meaning.

2. What is Word2Vec?

Word2Vec is a neural network model used to generate word embeddings.

3. What is the purpose of word vectors?

They help computers understand relationships between words.

4. What is vector arithmetic in NLP?

It is performing mathematical operations on word vectors to find semantic relationships.

5. Explain: $\text{King} - \text{Man} + \text{Woman} = ?$

The result is approximately "Queen".

6. What are semantic relationships?

They are meaning-based relationships between words.

7. What is cosine similarity?

It measures similarity between two vectors.

8. Name two pre-trained embedding models.

Word2Vec and GloVe.

9. What is CBOW?

CBOW predicts a word using surrounding context words.

10. What is Skip-gram?

Skip-gram predicts surrounding words using a target word.

2. Visualization of Word Embeddings

1. What is dimensionality reduction?

It reduces high-dimensional data into fewer dimensions.

2. What is PCA?

Principal Component Analysis is a technique for dimensionality reduction.

3. What is t-SNE?

t-SNE is a visualization method for high-dimensional data.

4. Why visualize embeddings?

To understand word relationships and clusters.

5. What do clusters represent?

Groups of semantically similar words.

6. Why are similar words close together?

Because their meanings are related.

7. Which library is used for plotting?

Matplotlib.

8. What is feature reduction?

Reducing the number of variables/features in data.

9. What are high-dimensional vectors?

Vectors containing many numerical values/features.

10. Why use PCA before visualization?

To convert vectors into 2D or 3D for plotting.

3. Custom Word2Vec Model

1. Why train a custom Word2Vec model?

To capture domain-specific meanings.

2. What is a corpus?

A collection of text documents.

3. What is tokenization?

Splitting text into words or tokens.

4. What is domain-specific data?

Data related to a specific field like medicine or law.

5. Which library is used?

Gensim.

6. What is vector_size?

The size of the word embedding vector.

7. What is window size?

The number of nearby words considered as context.

8. What is min_count?

Minimum frequency required for a word to be included.

9. Why are medical embeddings useful?

They understand medical terminology better.

10. How does Word2Vec learn?

By analyzing neighboring words in sentences.

4. Prompt Enrichment using Embeddings

1. What is prompt engineering?

Designing effective prompts for AI models.

2. Why enrich prompts?

To generate more detailed responses.

3. How do embeddings improve prompts?

They add semantically related words.

4. What is semantic similarity?

Similarity based on meaning.

5. What is text generation?

AI creating human-like text.

6. Which model was used?

GPT-2.

7. What is GPT-2?

A transformer-based text generation model.

8. Why compare prompts?

To observe improvement in output quality.

9. What is contextual relevance?

How closely the response matches the topic.

10. What are generative AI models?

Models that generate text, images, or other content.

5. Creative Text Generation

1. What is a seed word?

The starting input word.

2. How are similar words generated?

Using word embeddings.

3. What is creative AI writing?

AI-generated stories or sentences.

4. What is semantic generation?

Generating content based on meaning.

5. How do embeddings help storytelling?

They provide contextually related words.

6. What is NLP text generation?

Automatic generation of meaningful text.

7. Why use similar words?

To make content richer and coherent.

8. What is contextual sentence creation?

Generating sentences relevant to a topic.

9. What is language modeling?

Predicting words in a sequence.

10. What is automated content generation?

AI automatically producing text content.

6. Sentiment Analysis

1. What is sentiment analysis?

Identifying emotions in text.

2. What are positive sentiments?

Texts expressing happiness or satisfaction.

3. What are negative sentiments?

Texts expressing sadness or dissatisfaction.

4. Which library is used?

Transformers library.

5. What is a transformer model?

A deep learning architecture used in NLP.

6. What is Hugging Face?

A platform providing pre-trained AI models.

7. What is a pipeline?

A ready-made NLP workflow.

8. Give one application.

Customer review analysis.

9. What is opinion mining?

Extracting opinions from text.

10. What type of NLP task is sentiment analysis?

Text classification.

7. Text Summarization

1. What is text summarization?

Reducing long text into short meaningful text.

2. What is extractive summarization?

Selecting important sentences directly from text.

3. What is abstractive summarization?

Generating new summarized sentences.

4. Which library is used?

Transformers.

5. Why is summarization useful?

It saves reading time.

6. What is max_length?

Maximum length of summary.

7. What is min_length?

Minimum length of summary.

8. Give one application.

News summarization.

9. What is a pre-trained model?

A model already trained on large datasets.

10. How does AI summarize?

By understanding important information in text.

8. LangChain and Cohere

1. What is LangChain?

A framework for building LLM applications.

2. What is Cohere?

An AI platform providing language models.

3. Why use API keys?

To securely access AI services.

4. What is a prompt template?

A structured format for prompts.

5. What is an LLM?

Large Language Model.

6. What is document loading?

Reading documents into the program.

7. What is PDF processing?

Extracting text from PDFs.

8. What is chain execution?

Connecting multiple AI tasks sequentially.

9. What is prompt formatting?

Structuring prompts properly.

10. What are language models?

Models trained to understand and generate language.

9. Pydantic and Output Parsing

1. What is Pydantic?

A Python library for data validation.

2. Why use schemas?

To maintain structured output.

3. What is structured output?

Output in a fixed organized format.

4. What is output parsing?

Converting AI output into structured data.

5. What is Wikipedia API?

A tool to fetch Wikipedia information.

6. What is data validation?

Checking if data follows rules.

7. What is JSON?

JavaScript Object Notation used for structured data.

8. What is BaseModel?

The main class in Pydantic schemas.

9. Why is structured data important?

It improves readability and processing.

10. What is information extraction?

Extracting useful details from text.

10. Indian Penal Code Chatbot

1. What is a chatbot?

A program that interacts with users conversationally.

2. What is RAG?

Retrieval-Augmented Generation combines retrieval and generation.

3. What is FAISS?

A library for fast similarity search.

4. What are embeddings in chatbots?

Vector representations of text.

5. Why split documents into chunks?

To process large documents efficiently.

6. What is similarity search?

Finding text similar to a query.

7. What is vector storage?

Storing embeddings in vector databases.

8. What is question answering?

Answering user queries from documents.

9. Which embedding model was used?

all-MiniLM-L6-v2.

10. What are legal AI assistants?

AI systems that help with legal information.
